



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

CEF-2113 – Algorithms and Data Structures Lab Laboratory Manual

Department	:	School of Basic Science, Humanities and Management										
Course Name & code	:	CEF 2113 & Algorithms and Data Structures Lab										
Semester & branch	:	III Sem & BTech Computer Science & Finance Technology										
Name of the faculty	:	Dr. Manjunatha										
No of contact hours/week:		<table><tr><td>L</td><td>T</td><td>P</td><td>C</td></tr><tr><td>0</td><td>0</td><td>3</td><td>1</td></tr></table>	L	T	P	C	0	0	3	1		
L	T	P	C									
0	0	3	1									

CONTENTS

Lab No.	Title	Page No
	Course Objectives	
	Evaluation Plan	
	Instructions to Students	
1	Mapping of 2-D arrays to 1-D arrays	
2	Solving problems using recursion	
3	Stacks and their applications	
4	Queues and their applications	
5	Singly linked list and its applications	
6	Doubly linked list and circular linked list operations	
7	Trees and their applications	
8	Graph representations and their applications	
9	Graphs traversal algorithms	
10	Algorithm design Technique: Brute force method	
11	Algorithm design Technique: Divide and conquer method	
12	End Sem	

Course Objectives ·

1. Design, develop, and implement programs using the concepts of Recursion and the mapping of 2D arrays to 1D arrays
2. Design, develop, and implement programs and applications using linear data structures such as Stacks, Queues, and Linked Lists
3. Design, Develop, and Implement programs and applications using the non-Linear Data structures such as trees and graphs
4. Application of different algorithm design techniques .

Evaluation plan : (Tentative)

- Internal Assessment Marks: 60%
- End-semester assessment of 2-hour duration: 40 %

Evaluation pattern

Internal Marks – 60 + End Sem - 40	
Internal Marks	Internal Assessment – 40 + Mid-sem - 20
Internal Assessment	Lab observations – 2*11(22) + Viva (18)
Lab Observation	Record (1*11) + Execution (1*11)
Viva	Quiz(18)

INSTRUCTIONS TO THE STUDENTS

Pre-Lab Session Instructions

- Be on time, adhere to the institution's rules, and maintain decorum.
- Leave your mobile phones, pen drives, and other electronic devices in your bag and keep the bag in the designated place in the lab.
- Must Sign in to the log register provided.
- Make sure to occupy the allotted system and answer the attendance.

In-Lab Session Instructions

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- Copy the program and results for the lab record.
- Prescribed textbooks and class notes can be kept ready for reference if required.

General Instructions for the Exercises in Lab

- Academic honesty is required in all your work. You must solve all programming assignments independently, except where group work is authorized. This means you must not take, show, give, or otherwise allow others to take your program code, problem solutions, or other work.
- The programs should meet the following criteria:
 - Programs should be interactive with appropriate prompt messages, error messages if any, and descriptive messages for outputs.
 - Programs should perform input validation (Data type, range error, etc.), give appropriate

error messages, and suggest corrective actions.

- Comments should be used to give the statement of the problem, and every function should indicate the purpose of the function, inputs, and outputs.
- Statements within the program should be properly indented.
- Use meaningful names for variables and functions.
- Make use of constants and type definitions wherever needed.
- The exercises for each week are divided into three sets:
 - Solved solutions
 - Lab exercises - to be completed during lab hours
 - Additional Exercises - to be completed outside the lab or in the lab to enhance the skill

Questions for lab tests and examinations are not necessarily limited to the questions in the manual but may involve some variations and/or combinations of the questions.

THE STUDENTS SHOULD NOT

- Possess mobile phones or any other electronic gadgets during lab hours.
- Go out of the lab without permission.
- **Change/update any configuration in your allotted system. If so, the student will lose internal assessment marks**

Week 1 - Mapping of 2-D Arrays to 1-D Arrays in Java

Learning Objectives

After completing this experiment, students will be able to:

1. Understand the memory representation of 2-D arrays.
2. Convert a 2-D array into a 1-D array using Row-Major Mapping.
3. Convert a 2-D array into a 1-D array using Column-Major Mapping.
4. Access elements using mapping formulas.
5. Apply array mapping concepts in data structure applications.

Two-Dimensional Array

A two-dimensional array consists of rows and columns.

Example:

```
1 2 3
4 5 6
7 8 9
```

Declaration:

```
int[][] arr =
{
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
```

Why Mapping is Required?

Computer memory is linear in nature. Therefore, multidimensional arrays are stored internally as one-dimensional sequences.

Mapping converts a 2-D array into a 1-D array representation.

Example:

2-D Array

```
1 2 3
4 5 6
7 8 9
```

1-D Representation

1 2 3 4 5 6 7 8 9

Row-Major Mapping

In Row-Major Mapping, elements are stored row by row.

Formula

For an element at position:

$$A[i][j]$$

The corresponding index in the 1-D array is:

$$\text{Index} = i \times \text{Number_of_Columns} + j$$

Example

For

1 2 3

4 5 6

7 8 9

Mapping becomes:

1 2 3 4 5 6 7 8 9

Column-Major Mapping

Formula

$$\text{Index} = j \times \text{Number_of_Rows} + i$$

Example

For

1 2 3

4 5 6

7 8 9

Mapping becomes:

1 4 7 2 5 8 3 6 9

Algorithm (Row-Major Mapping)

1. Read the number of rows and columns.
2. Create a 2-D array.
3. Accept elements from the user.
4. Create a 1-D array of size rows $\times$ columns.
5. Traverse the 2-D array row-wise.
6. Store elements into the 1-D array.
7. Display the mapped array.

Sample Program

Program: Convert 2-D Array into 1-D Array Using Row-Major Mapping

```
import java.util.Scanner;

public class RowMajorMapping
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();
        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();
        int[][] arr = new int[rows][cols];
        int[] oneD = new int[rows * cols];
        System.out.println("Enter array elements:");
        for(int i = 0; i < rows; i++)
        {
            for(int j = 0; j < cols; j++)
            {
                arr[i][j] = sc.nextInt();
            }
        }
    }
}
```

```
int k = 0;
for(int i = 0; i < rows; i++)
{
    for(int j = 0; j < cols; j++)
    {
        oneD[k++] = arr[i][j];
    }
}
System.out.println("Mapped 1-D Array:");
for(int value : oneD)
{
    System.out.print(value + " ");
}
}
```

Sample Output

Enter number of rows: 3

Enter number of columns: 3

Enter array elements:

1 2 3

4 5 6

7 8 9

Mapped 1-D Array:

1 2 3 4 5 6 7 8 9

Week 1: Exercises:

1. Write a Java program to convert a 2-D array into a 1-D array using Column-Major Mapping.

Expected Output

Original Array

1 2 3

4 5 6

7 8 9

Column Major Mapping

1 4 7 2 5 8 3 6 9

2. Write a Java program to calculate the 1-D index position of an element present at row i and column j using the Row-Major Mapping formula.

Example

Rows = 4

Columns = 5

Element Position = (2,3)

Index = $2 \times 5 + 3$

Index = 13

Expected Output

Mapped Index = 13

3. Write a Java program to convert a given 1-D array back into a 2-D array of specified dimensions.

Example Input

1-D Array:

10 20 30 40 50 60

Rows = 2

Columns = 3

Expected Output

10 20 30

40 50 60

4. Store two matrices in 1-D arrays using Row-Major Mapping and perform matrix addition.

Example

Matrix A

1 2

3 4

Matrix B

5 6

7 8

Expected Output

Sum Matrix

6 8

10 12

5. Write a Java program to read a matrix and determine whether it is a sparse matrix (contains more zeros than non-zero elements). Store all non-zero elements in a 1-D array using the format: row column value

Example Input

0 0 3

0 5 0

7 0 0

Expected Output

Sparse Matrix Representation

0 2 3

1 1 5

2 0 7

6. Write a Java program to find an element in a 2-D array and display its corresponding 1-D mapped index using Row-Major Mapping.

Example Input

Matrix

10 20 30

40 50 60

70 80 90

Search Element: 60

Expected Output

Element Found at:

Row = 1

Column = 2

Mapped Index = 5

Formula Used

Index = (Row × Number_of_Columns) + Column

Additional Exercises:

7. Write a Java program to:
 - Read a matrix.
 - Store it in a 1-D array using Row-Major Mapping.
 - Generate the transpose of the matrix.
 - Display the transposed matrix.
8. Write a Java program to:
 - Read two matrices.
 - Store both matrices as 1-D arrays using Row-Major Mapping.
 - Perform matrix multiplication.
 - Display the resultant matrix.